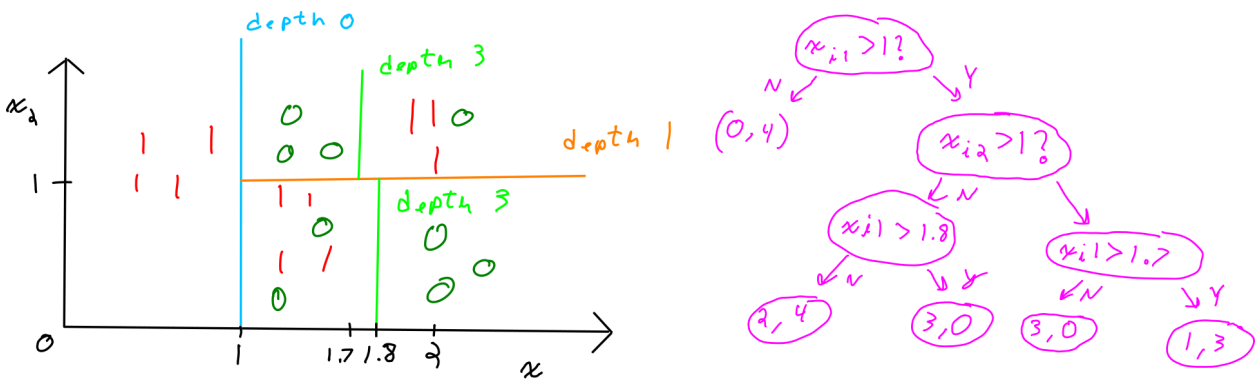
5 Decision Trees

Decision Trees are versatile machine learning algorithms that excel in various tasks including classification, regression, and even multi-output challenges. These algorithms are exceptionally powerful, capable of modeling complex datasets effectively. Furthermore, Decision Trees serve as the foundational building blocks for Random Forests, which stand as some of the most robust machine learning models currently in use. Trained decision tree models can get quite big.

In this chapter, we will explore the essentials of Decision Trees, starting with their training, visualization, and prediction processes. We will then look into the CART (Classification and Regression Trees) training algorithm, which Scikit-Learn utilizes for constructing Decision Trees. Additionally, we will examine how to regulate the complexity of Decision Trees and adapt them for regression tasks. The chapter concludes by addressing some inherent limitations of Decision Trees.



(a) (b)

Figure 5.1: The basic connect of a decision tree, showing (a) how a decision tree is built, and (b) the developed decision tree.

5.1 Decision Tree Classification

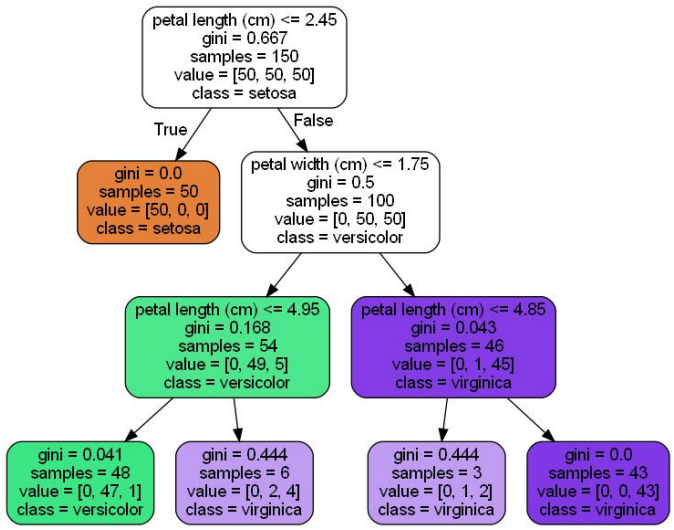


Figure 5.2: Iris decision tree.

A decision tree for the Iris Dataset looks like Figure 5.2. Let us examine how the Decision Tree processes predictions. Imagine you discover an iris flower and want to classify it. Starting at the root node (depth 0, at the top), the node queries if the petal length of the flower is less than 2.45 cm. If so, you proceed to the root's left child node (depth 1, left), which is a leaf node, meaning it poses no further questions. You would then refer to the predicted class of that node, and the Decision Tree identifies your flower as an Iris-Setosa (class=setosa).

Consider finding another iris with a petal length exceeding 2.45 cm. This time, you move to the root's right child node (depth 1, right). Since this node is not a leaf, it inquires if the petal width is less than 1.75 cm. If affirmative, your flower is likely an Iris-Versicolor (depth 2, left); otherwise, it is probably an Iris-Virginica (depth 2, right). The process is straightforward.

The attributes of a node include:

- *class* is the classification of the leaf.
- *samples* indicates the count of training instances affected. For instance, 100 training instances have a petal length greater than 2.45 cm (depth 1, right), of which 54 have a petal width smaller than 1.75 cm (depth 2, left).
- *value* shows the count of training instances for each class at this node: for example, the bottom-right node affects 0 Iris-Setosa, 1 Iris-Versicolor, and 45 Iris-Virginica.
- *gini* is a measure of impurity: a node is considered 'pure' (gini=0) if all training instances it affects belong to the same class. For example, the depth-1 left node exclusively pertains to Iris-Setosa instances, rendering it pure with a gini score of 0.

The Gini impurity G_i for the ith node is computed using Equation 5.1. For instance, the Gini score for the depth-2 left node is calculated as follows: $1-(0/54)^2-(49/54)^2-(5/54)^2 \approx 0.168$. We

will discuss an alternative impurity measure later.

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2 \tag{5.1}$$

where $p_{i,k}$ is the proportion of class k instances among the training instances at the ith node.

Figure 5.3 illustrates the decision boundaries of the Decision Tree. The bold vertical line denotes the decision boundary for the root node (depth 0): petal length = 2.45 cm. The left region is pure (comprising only Iris-Setosa) and cannot be further split. However, the right region is impure, prompting the depth-1 right node to divide it at a petal width of 1.75 cm, indicated by the dashed line. With `max_depth` set to 2, the tree does not extend further. Increasing `max_depth` to 3 would lead the two depth-2 nodes to introduce additional decision boundaries, marked by the dotted lines.

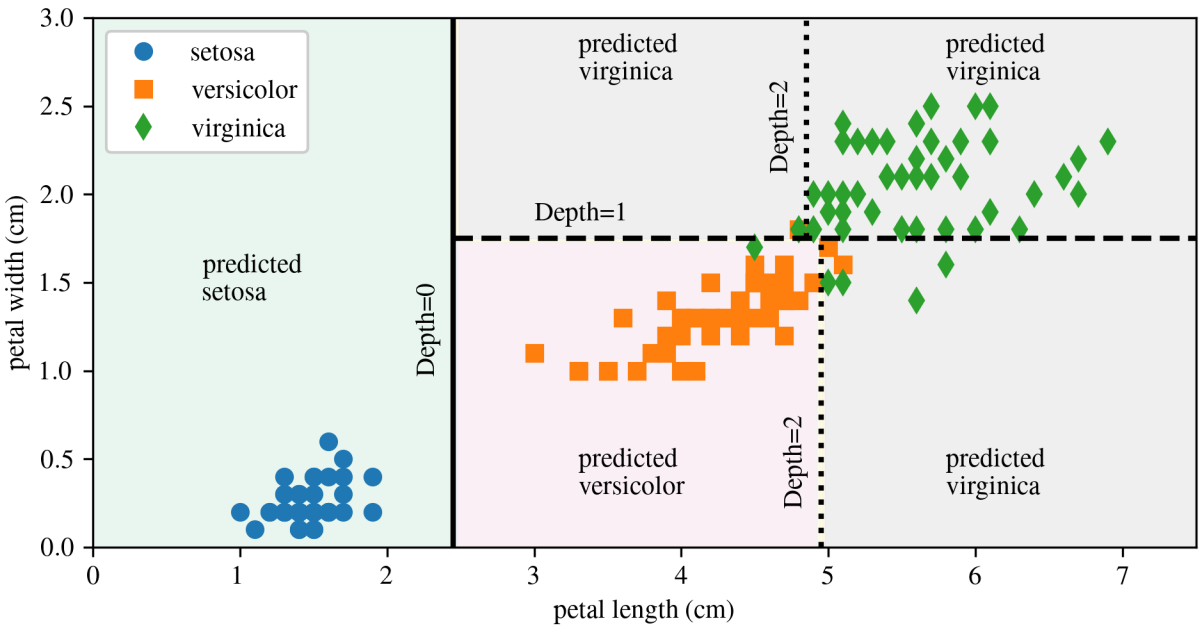


Figure 5.3: Decision Tree decision boundaries.

5.1.1 Estimating Class Probabilities

Decision Trees can estimate the probability that an instance belongs to a specific class k. It navigates the tree to find the leaf node for the instance and returns the proportion of class k training instances in that node. For example, consider a flower with petal dimensions of 5 cm in length and 1.5 cm in width. It reaches the depth-2 left node, where the Decision Tree would assign probabilities of 0% for Iris-Setosa (0/54), 90.7% for Iris-Versicolor (49/54), and 9.3% for Iris-Virginica (5/54). If asked for a class prediction, it outputs Iris-Versicolor (class 1), which has the highest probability.

Interestingly, the same estimated probabilities apply throughout the bottom-right rectangle of Figure 5.3, even in a scenario where the petal dimensions are 6 cm by 1.5 cm where one might intuitively expect an Iris-Virginica classification.

Example 5.1 Decision Tree Classifier
This example uses Scikit-Learn?s `DecisionTreeClassifier` to classify iris species based on petal features. The decision process is visualized using Graphviz, showing the structure of learned rules and how decisions are made based on feature thresholds. This online viewer is the easiest way to do that https://dreampuf.github.io/GraphvizOnline/

5.2 The CART Training Algorithm

Scikit-Learn employs the Classification And Regression Tree (CART) algorithm to develop Decision Trees [a], commonly referred to as "growing" trees. The procedure is straightforward: the algorithm splits the training set into two parts using one feature k and a threshold t_k (e.g., "petal length $\leq$ 2.45 cm"). It selects the feature and threshold by searching for the pair (k, t_k) that yields the most homogeneous subsets, measured by the subsets' size.

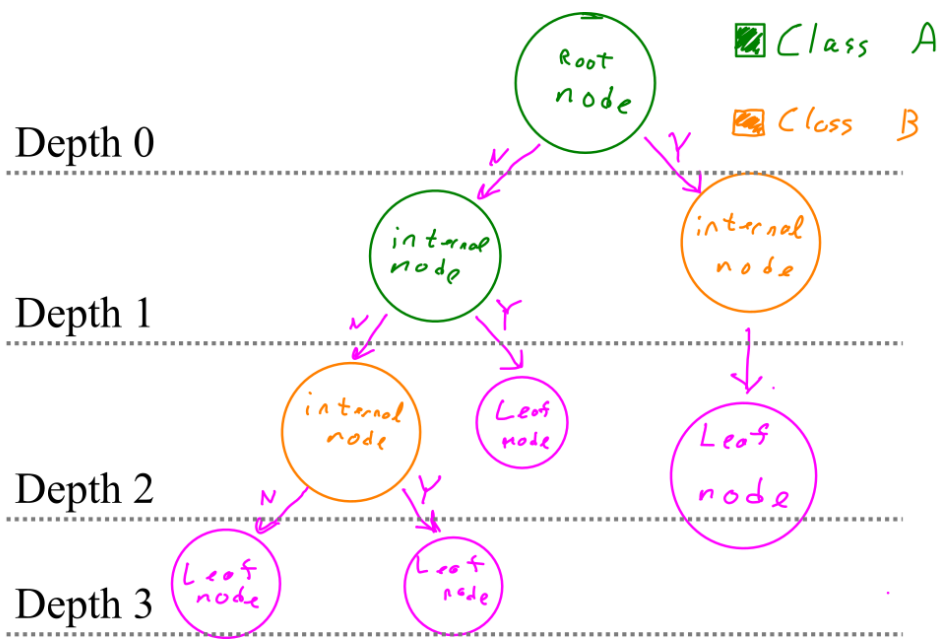


Figure 5.4: Illustration of the CART algorithm process.

The objective function it aims to minimize during this process is shown in Equation 5.2.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}} \tag{5.2}$$

$$\text{where} \quad \begin{cases} G_{\text{left/right}} \text{ measures the impurity of the left/right subset,} \\ m_{\text{left/right}} \text{ is the number of instances in the left/right subset.} \end{cases}$$

After the initial split, the algorithm continues to divide the resulting subsets and their subsequent divisions recursively. This recursive process halts when it reaches the predefined maximum depth

[a] Breiman, Leo. Classification and regression trees. Routledge, 2017

(`max_depth`) set through a hyperparameter, or if no further impurity-reducing splits can be found.

Unlike linear models that assume a specific data structure, Decision Trees impose few assumptions on their initial data. If not appropriately constrained, a Decision Tree can intricately conform to the training data, leading to overfitting. This model type is described as nonparametric, not due to a lack of parameters, which it can have in abundance, but because the parameters are not predetermined before training, allowing the model's structure to freely mirror the data intricacies. Conversely, a parametric model, like a linear model, has a fixed number of parameters, limiting its flexibility but also minimizing overfitting risks while potentially increasing underfitting risks.

To mitigate overfitting in Decision Trees, it is essential to control the model's freedom during training through regularization. The regularization hyperparameters vary by the algorithm, but typically, the tree's maximum depth can be restricted. In Scikit-Learn, this is managed by the `max_depth` hyperparameter, which is unlimited by default. Lowering `max_depth` helps regularize the model, thereby reducing overfitting likelihood.

Other parameters in Scikit-Learn's DecisionTreeClassifier also influence the tree's structure:

- `min_samples_split`: The minimum number of samples required to split a node.
- `min_samples_leaf`: The minimum number of samples a leaf node must have.
- `min_weight_fraction_leaf`: Similar to `min_samples_leaf`, but expressed as a fraction of total weighted instances.
- `max_leaf_nodes`: The maximum number of leaf nodes.
- `max_features`: The maximum number of features evaluated for splitting at each node.

Adjusting these parameters by increasing `min_*` values or decreasing `max_*` values will help regularize the model.

Figure 5.5 illustrates two Decision Trees trained on the moons dataset: one on the left with default hyperparameters (unrestricted) and another on the right with `min_samples_leaf=4`. The left model appears to be overfitting, whereas the right model, with its restrictions, likely offers better generalization.

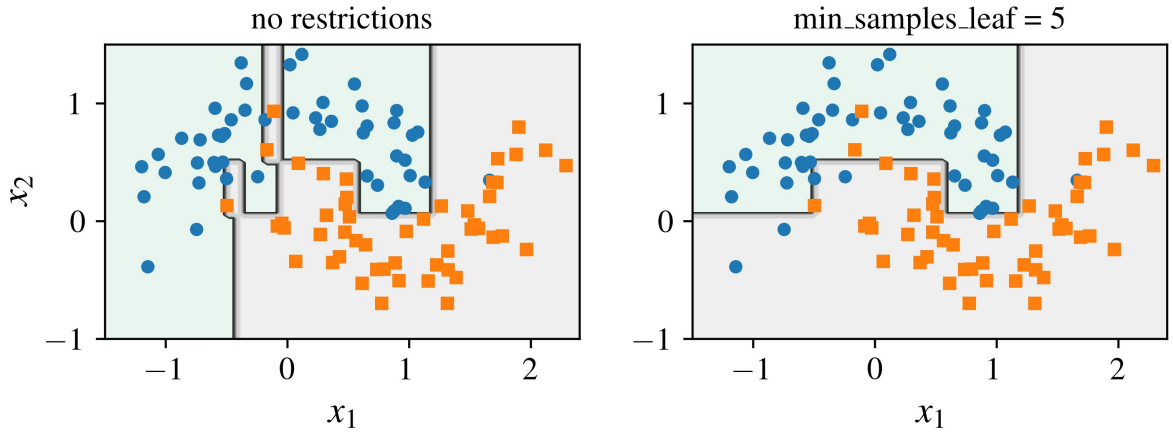


Figure 5.5: Regularization effects using `min_samples_leaf`

5.2.1 Computational Complexity

Making predictions with a Decision Tree involves traversing from the root to a leaf. Decision Trees are generally well-balanced, implying that the path from the root to a leaf involves crossing approximately $O(\log_2(m))$ nodes. Since checking a node involves examining just one feature, the overall complexity for making predictions is $O(\log_2(m))$, making it exceptionally fast regardless of the feature count.

Training, however, is more intensive: the algorithm examines all features (or fewer if `max_features` is set) across all samples at each node, leading to a training complexity of $O(n \times m\log(m))$. For smaller datasets (few thousand instances or fewer), Scikit-Learn can expedite training by presorting data (set `presort=True`), though this method becomes inefficient for larger datasets.

5.2.2 Gini Impurity or Entropy?

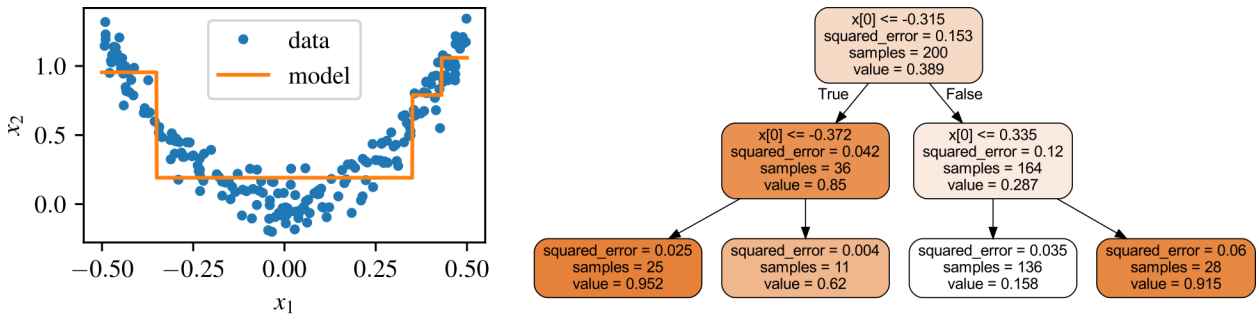
Decision Trees can use the Gini impurity as a default criterion for node purity, but entropy can also be utilized by setting the criterion hyperparameter to "entropy". Originally a thermodynamic concept representing molecular disorder, entropy reaches zero when molecules are in a complete state of rest and order. This concept was later adopted in information theory, introduced by Shannon, to quantify the average informational content in messages entropy is zero when all messages are identical. In the realm of Machine Learning, entropy serves as a measure of impurity: it is zero in a dataset if all its elements belong to a single class. The entropy for the ith node is defined as shown in Equation 5.3.

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2(p_{i,k}) \tag{5.3}$$

The choice between Gini impurity and entropy often results in negligible differences, producing similar trees. Gini impurity has a slight computational advantage and is thus the default choice. However, it tends to separate the most frequent class into a distinct branch, whereas entropy generally yields more balanced trees.

5.3 Decision Tree Regression

Decision Trees are not limited to classification tasks; they can also be adapted for regression. To illustrate, we use Scikit-Learn's `DecisionTreeRegressor` to train a regression tree on a noisy quadratic dataset with `max_depth=2`. The structure of this tree is displayed in Figure 5.6.



(a) (b)

Figure 5.6: A regression model developed using a Decision Tree, showing the: (a) model superimposed over noisy data, and (b) the decision tree developed for the task.

The mechanics of this regression tree are similar to those of a classification tree. However, instead of predicting a class at each node, the tree predicts a continuous value. For instance, if we need to predict the output for a new instance where $x_1 = 0.6$, we traverse the tree from the root to a leaf that predicts a value of 0.1106. This value represents the average target value of the 110 training instances corresponding to that leaf, achieving a Mean Squared Error (MSE) of 0.0151 across these instances.

The predictions from this model are visualized in Figure 5.7, with results shown for trees of depth 2 and 3. The deeper tree partitions the input space into more regions, with each region's predicted value being the average target value of the instances it encompasses. The model attempts to organize the regions such that the instances within each are as close as possible to their predicted value.

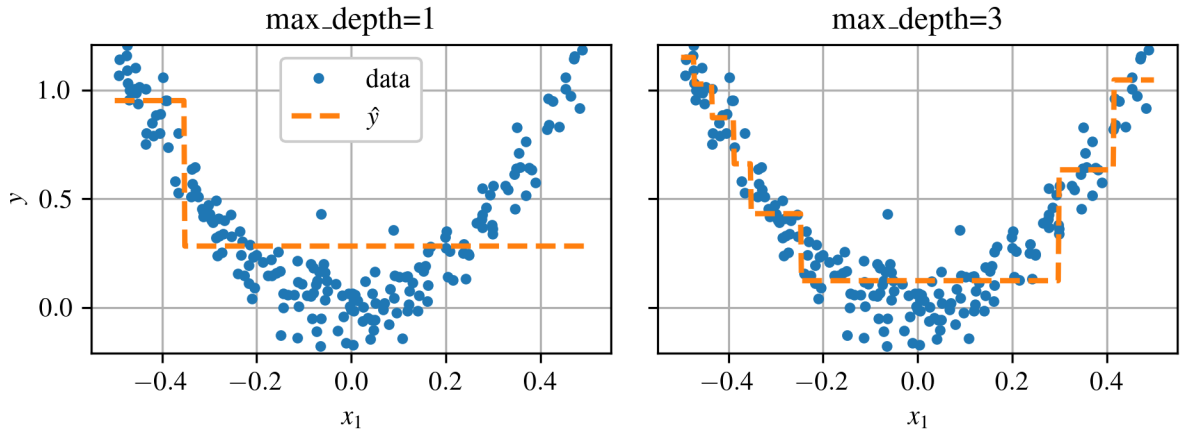


Figure 5.7: Comparison of predictions from two Decision Tree regression models with varying depths.

The CART algorithm for regression trees aims to minimize the MSE when splitting the training set, similar to how it minimizes impurity in classification tasks. The cost function minimized by the algorithm is represented by Equation 5.4.

$$J(k,t_k) = \frac{m_{\text{left}}}{m}\text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m}\text{MSE}_{\text{right}} \quad \text{where} \quad \begin{cases} \text{MSE}_{\text{node}} = \sum\limits_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)}\right)^2 \\ \hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum\limits_{i \in \text{node}} y^{(i)} \end{cases} \tag{5.4}$$

Similar to classification, Decision Trees for regression can overfit if not properly regularized. Without regularization, the predictions, as depicted on the left of Figure 5.8, can fit the training data excessively. By setting `min_samples_leaf` to 10, a more generalized model is achieved, as shown on the right in the same figure.

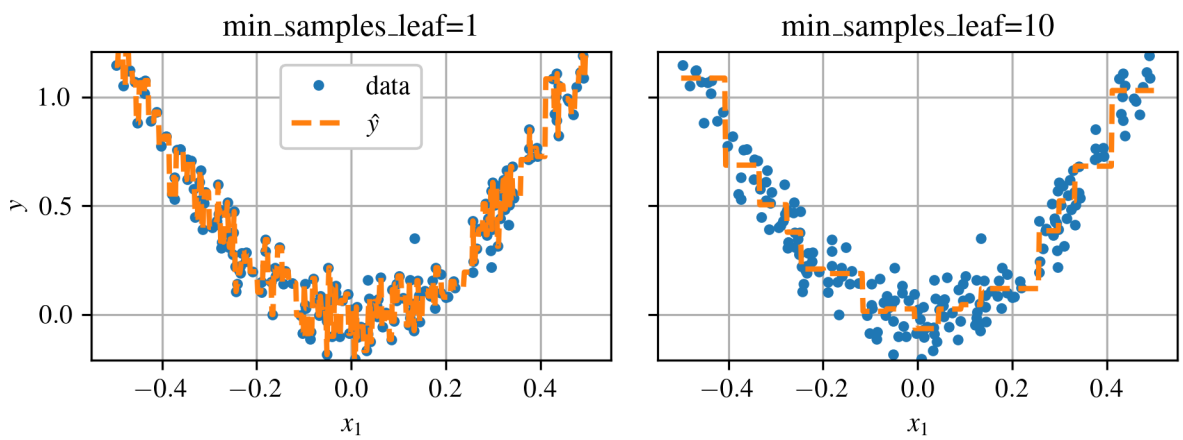


Figure 5.8: Impact of regularization on Decision Tree regression models.

Example 5.2 Decision Tree Regression
This example uses Scikit-Learn's `DecisionTreeRegressor` to fit a non-linear relationship between input and output data. A tree of depth 4 is trained on noisy quadratic data and visualized using Graphviz to show how the regression model partitions the input space.

5.4 Instability

While Decision Trees offer simplicity, interpretability, versatility, and power, they come with certain drawbacks. A notable limitation is their preference for orthogonal decision boundaries, which makes them highly sensitive to the orientation of the data. For instance, Figure 5.9 illustrates how a simple linearly separable dataset can be perfectly split by a Decision Tree in its original alignment, whereas a 45° rotation results in a convoluted decision boundary. Despite perfect fits to their respective training sets, the rotated tree's model is likely to perform poorly on unseen data. Utilizing Principal Component Analysis (PCA) can often mitigate this issue by reorienting the data more suitably.

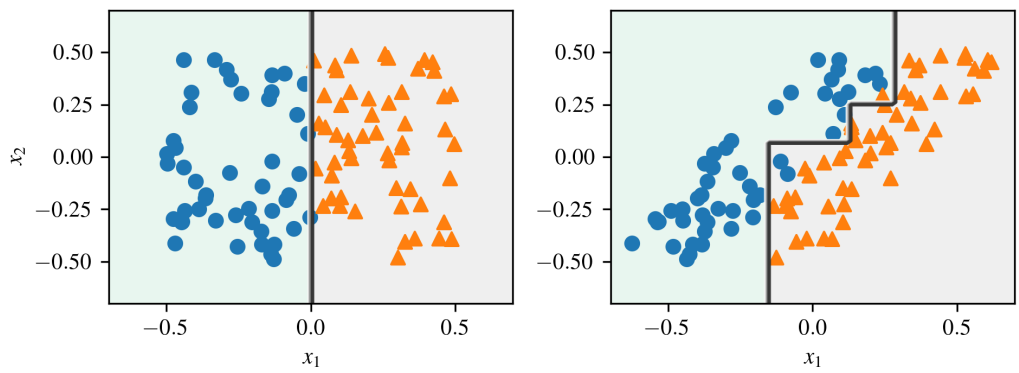


Figure 5.9: Sensitivity to training set rotation.

More broadly, Decision Trees are sensitive to small variations in training data. For instance, changing the seed of the random number generator can lead to a substantially different model, as shown in Figure 5.10. This variability is partly due to the stochastic nature of the Scikit-Learn's training algorithms; different models may result from the same data unless the `random_state` hyperparameter is fixed.

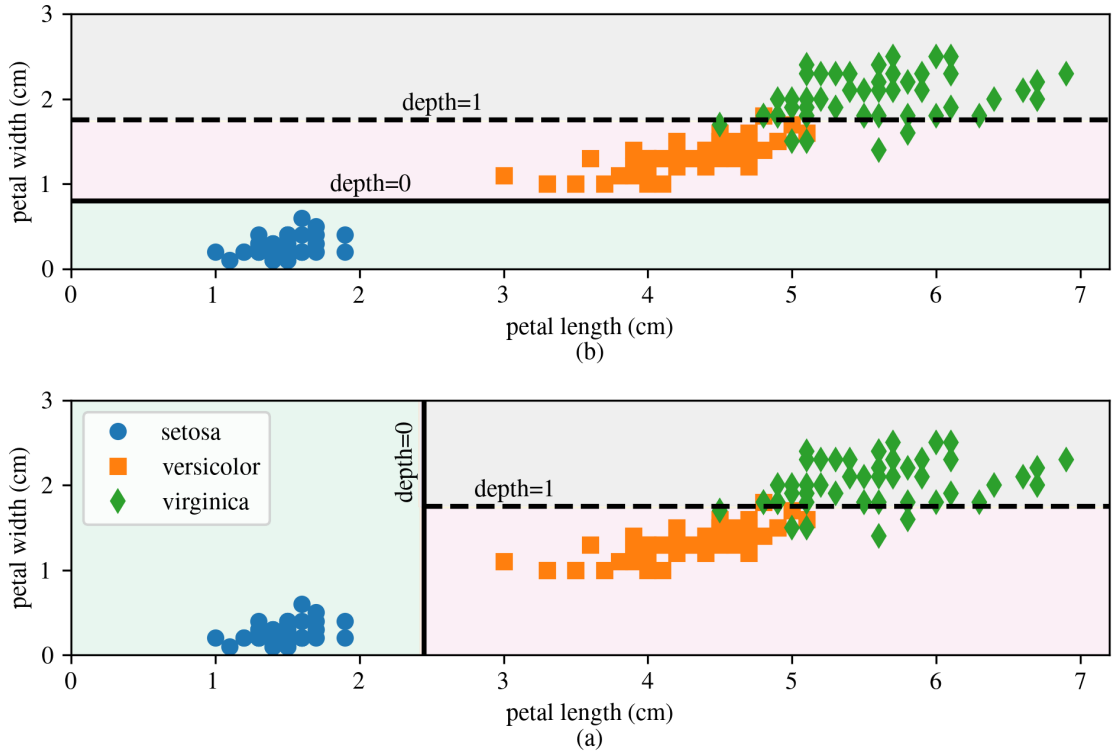


Figure 5.10: Decision Tree Sensitivity to initial conditions, showing: (a) random number generator seeded with "1", and: (b) random number generator seeded with "2". [a]

[a] "2" is the author's preferred random number seed.

The next section introduces Random Forests, which overcome these instabilities by averaging predictions across multiple trees, enhancing the robustness and generalizability of the model.

5.5 Random Forest

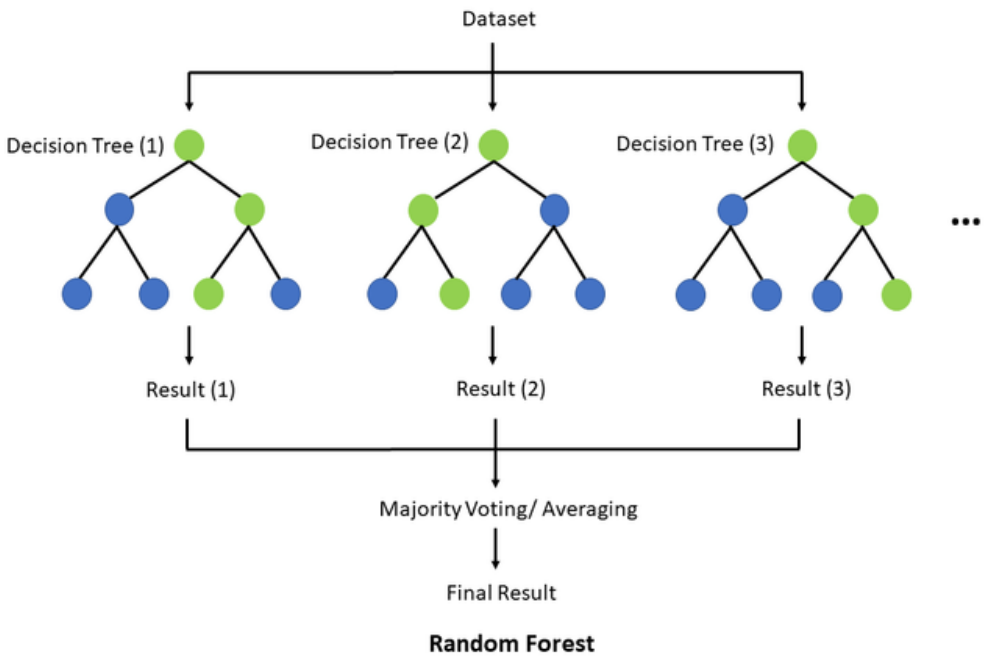


Figure 5.11: random forest with majority voting. [a]

Random forests, also known as random decision forests, are an ensemble learning method used for classification, regression, and other tasks. This method constructs multiple decision trees during training. For classification, the output of the random forest is the class chosen by the majority of the trees. For regression tasks, it returns the mean or average prediction from the individual trees. Random decision forests address the tendency of decision trees to overfit their training set. Generally, random forests perform better than individual decision trees, though they may not reach the accuracy of gradient boosted trees. The performance of random forests can vary depending on the characteristics of the data.

[a] TseKiChun, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons